Fire & Smoke Detection for Industrial Cameras

A DINOv3-based detector with an image-level classifier
Training methodology, two-stage results and evaluation

Backbone	DINOv3 ViT-S/16 (vit_small_patch16_dinov3.lvd1689m)
Detection head	Faster R-CNN over a 4-level feature pyramid
Second head	Image-level scene classifier + glow prior
Dataset	D-Fire - 21,527 images, 45.7% verified negatives
Training	Kaggle P100, two stages, 55 epochs total
Best test mAP@0.5	0.7261
False-alarm rate	0.0070 at confidence 0.90
Report generated	2026-09-09

1. Executive summary

This project detects fire and smoke on a fixed industrial camera that runs continuously. Rather than training a detector from scratch, it builds on **DINOv3**, a vision transformer trained self-supervised on 1.7 billion images, and keeps that backbone frozen so its general visual representation survives contact with a comparatively small 21,527-image dataset. Two lightweight heads are trained on top: a Faster R-CNN detection head that draws boxes, and an image-level classifier that also receives hand-crafted illumination statistics so it can respond to fire that is hidden from the camera and visible only as light cast on surrounding surfaces.

Training ran in two stages. Stage 1 trained the heads against a completely frozen trunk. Stage 2 took those weights and released the last two transformer blocks at a much lower learning rate. The headline outcome is below; everything in this document is generated directly from the run artifacts.

Metric	Stage 1	Stage 2	Change
Validation mAP@0.5	0.7312	0.7406	+0.0094
Test mAP@0.5	0.7196	0.7261	+0.0065
Test AP50 - smoke	0.8050	0.8146	+0.0096
Test AP50 - fire	0.6342	0.6377	+0.0035
Fire recall at 0.7% false-alarm rate	0.7587	0.7803	+0.0216

The number that matters for deployment is the pair, not the mAP alone. On the 4,306-image test split the model reaches mAP@0.5 = 0.7261 while raising a detection on only 14 of 2,005 verified-negative images at confidence 0.90 - a 0.70% false-alarm rate. An accuracy figure quoted without its false-alarm rate says nothing about whether a system can be left switched on.

2. Requirements that shaped the design

#	Requirement	How the design answers it
1	Camera at height; poor industrial lighting, but fire is self-luminous	Highlight-preserving low-light augmentation on 35% of training images
2	Runs 24/7; false alarms must be very rare	False-alarm rate measured on 9,838 negatives; threshold chosen from that curve; temporal confirma
3	Fire may be hidden, visible only as light on nearby objects	Image-level classifier + glow prior + synthetic flame-occlusion augmentation
4	Visible smoke must also be detected	Smoke is a first-class class with its own AP and its own recall column
5	Same camera conditions apply to smoke	One augmentation pipeline covers both classes
6	Training on Kaggle with the uploaded dataset	Scripts auto-discover data.yaml; resume support for session timeouts
7	Images first, video later	Image training complete; video layer written and unit-tested
8	Must be explainable at any point	Living project log plus this report, both generated from run artifacts

3. Dataset

D-Fire (Gaia Solutions on Demand), YOLO-format bounding boxes with class 0 = smoke and 1 = fire. An empty label file marks a *verified negative* - an image confirmed to contain neither. Those negatives are 45.7% of the dataset and they are the reason the false-alarm requirement can be measured at all rather than merely asserted.

Split	Images	Positives	Negatives	Smoke boxes	Fire boxes
train	13,776	7,553	6,223	7,660	9,659
val	3,445	1,835	1,610	1,890	2,155
test	4,306	2,301	2,005	2,311	2,878
total	21,527	11,689	9,838	11,854	14,685

Data issue found and handled. 18 label lines across the three splits describe zero-area boxes. TorchVision rejects an entire batch if one degenerate box reaches it, so the loader drops any box smaller than 2 px per side and the preparation script now strips them at source.

Domain gap, stated plainly. D-Fire is largely daytime, web-sourced, close-range imagery. The deployment target is an elevated, fixed, often dark industrial view. The augmentation pipeline narrows this gap; it does not close it. Site footage remains necessary for a final fine-tuning stage.

4. Model: what we are using and why

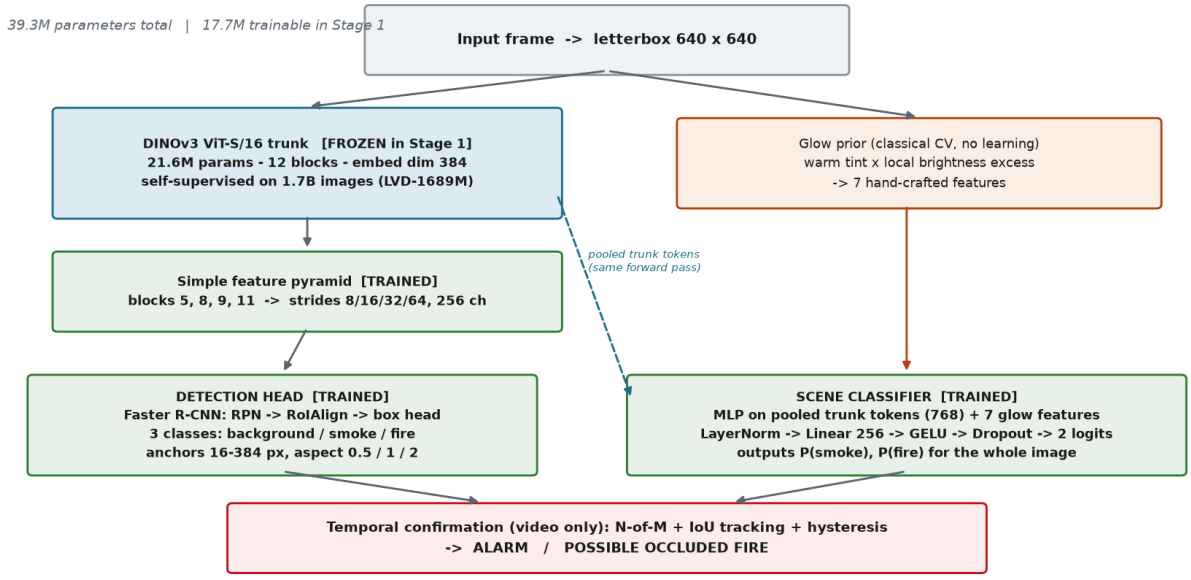

Figure 1 - Model architecture. Blue = frozen in Stage 1, green = trained, orange = classical computer vision with no learned parameters.

4.1 Why DINOv3 as the backbone

DINOv3 is a vision transformer trained self-supervised on 1.7 billion images. Its features are far more general than anything 14,000 training images could teach from scratch, and that matters here specifically because the real deployment domain - a dark factory at three in the morning - does not appear in the training set at all. The backbone was also a requirement from the project supervisor; it happens to be well matched to the problem.

The trunk is **frozen** by default, for three reasons:

- It preserves the pretrained representation instead of overwriting it with a small dataset.
- No gradients flow through the transformer, so activations are not stored - this is what lets 640 px at batch 8 fit comfortably on a Kaggle GPU.

- It is the honest reading of "use DINO": a strong frozen representation plus a light task head.

4.2 Why this classifier, and why there are two heads

The **detection head** is a Faster R-CNN head - a region proposal network, RoIAlign, and a box classifier - over a four-level feature pyramid. A plain ViT produces features at a single stride, so following ViTDet the pyramid is built by resampling four intermediate blocks to strides 8, 16, 32 and 64. This is what makes small, distant flames detectable; a single stride-16 map would miss them. Faster R-CNN was chosen over a one-stage head because its two-stage design gives better precision at high confidence, which is exactly the regime a low-false-alarm system operates in.

The **scene classifier** is a small MLP on the pooled trunk tokens concatenated with seven hand-crafted glow statistics. It exists for two deployment reasons that a box detector cannot serve:

- **Occluded fire.** A fire behind machinery has no flame-shaped object to box, only warm light spread across nearby surfaces. A whole-image classifier can still respond to that.
- **Cross-checking.** Two semi-independent opinions let the alarm layer demand agreement, which is the cheapest false-positive filter available.

Both heads share a **single** trunk forward pass - the backbone stashes its pooled tokens - so the second head costs almost nothing. It also turned out to be the stronger of the two: after Stage 2 the scene head reaches 0.952 recall on fire at roughly 2% false-positive rate.

The glow prior was measured, not assumed. Of its seven features only `warm_cast` is a strong standalone cue, and it *strengthens* in the dark (ROC AUC 0.73 → 0.83) - precisely where the box detector is weakest. The remaining glow statistics are informative but *inverted*: D-Fire negatives contain sunsets and warm street lighting that out-glow real fires. They are kept as classifier inputs, never thresholded directly. Three global-exposure features were deliberately **removed** despite scoring highest (AUC 0.82-0.84), because they encode "fire photographs tend to be dark" - a model leaning on that would alarm on nightfall every single night.

4.3 Architecture parameters

Component	Configuration
Trunk	DINOv3 ViT-S/16, 12 blocks, embed dim 384, 21.6M params
Pyramid source blocks	5, 8, 9, 11 (of 12)
Pyramid strides / channels	8 / 16 / 32 / 64, 256 channels each
Anchor sizes	(16, 32), (48, 96), (128, 192), (256, 384) px
Anchor aspect ratios	0.5, 1.0, 2.0
Detection classes	3 (background, smoke, fire)
Normalisation	LayerNorm throughout (no BatchNorm anywhere)
Scene head	LayerNorm -> Linear(775, 256) -> GELU -> Dropout 0.2 -> Linear(256, 2)
Glow features	7 (3 exposure features deliberately excluded)
Input size	640 x 640 letterboxed (multiple of 64)
Total parameters	39.3M (17.7M trainable in Stage 1)

LayerNorm rather than BatchNorm is a deliberate choice: with no running statistics anywhere in the model, small batch sizes stay safe and a validation loss can be computed in training mode without corrupting anything.

5. Training data pipeline

Augmentation is where most of the deployment requirements are actually addressed, because D-Fire is overwhelmingly bright, daytime, web-sourced imagery while the target is a fixed industrial camera running around the clock.

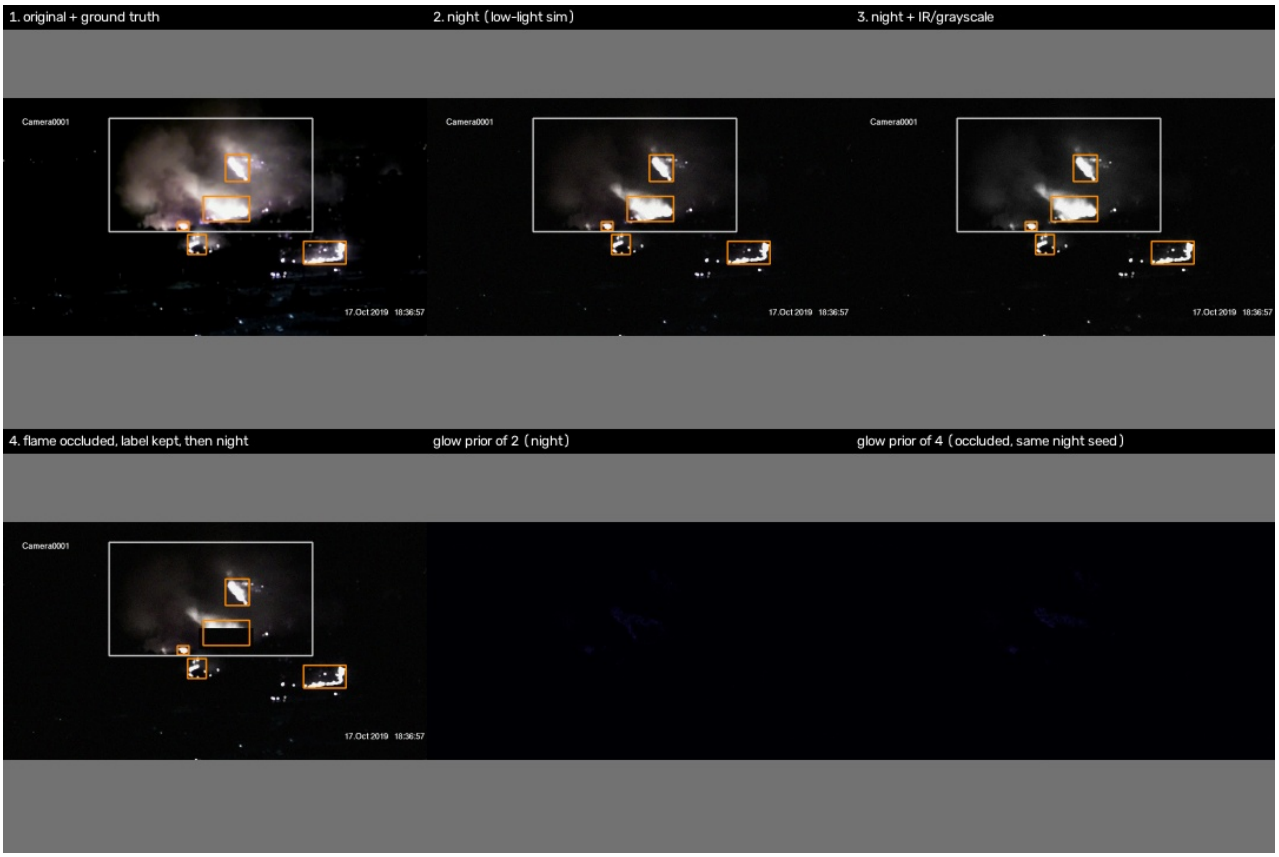

Figure 2 - The augmentation pipeline on a real D-Fire frame. Top: original with ground truth, low-light simulation, and night plus IR/grayscale. Bottom: a flame occluded by a synthetic obstacle with the label kept, and the glow prior computed on both night variants using the same random seed so they are directly comparable.

Augmentation	Rate	What it simulates and why
night	0.35	Auto-exposure closing down at night. Crucially it does NOT dim uniformly - the darkening factor is relaxed where a pixel is alr
gray	0.10	IR / night-mode cameras that drop colour entirely, forcing the model not to rely on the orange cue alone.
occlude_fire	0.25	Covers 45-85% of a flame box with a synthetic obstacle while keeping the label. D-Fire has essentially no examples of fire yo
crop	0.80	Scale and viewpoint variation; also generates additional negatives.
jitter	0.80	Brightness, contrast, saturation and small hue shifts.
hflip	0.50	Horizontal mirror.
blur	0.10	Motion blur.

Scene labels are recomputed *after* augmentation from the surviving boxes, so a crop that removes the only flame correctly flips the image-level label to negative.

6. Two-stage training

Figure 3 - How Stage 1 feeds Stage 2. Only the weights carry across; the optimizer, epoch counter and learning-rate schedule all restart.

The two stages are joined by `--init-from`, not `--resume`. The distinction matters: `--resume` restores the optimizer state, epoch counter and history, which is what you want after a session timeout, but for Stage 2 it would make the run believe it had already finished. `--init-from` loads only the weights and starts a clean run with a fresh cosine schedule.

Hyper-parameter	Stage 1	Stage 2
Initialised from	DINOv3 pretrained trunk	Stage 1 best.pt (--init-from)
Trunk blocks unfrozen	0 (fully frozen)	2 (last two)
Trainable parameters	17.7M	~32M
Head / neck learning rate	1e-4	5e-5
Trunk learning rate	n/a	2.5e-6 (5e-5 x 0.05)
Optimizer	AdamW, weight decay 1e-4	AdamW, weight decay 1e-4
LR schedule	500-iter warmup, then cosine to 1%	500-iter warmup, then cosine to 1%
Epochs	40	15
Image size / batch	640 / 8	640 / 8
Mixed precision	fp16 with GradScaler	fp16 with GradScaler
Gradient clipping	10.0	10.0
Scene-head loss weight	1.0	1.0
Model selection	validation mAP@0.5	validation mAP@0.5
Early-stop patience	8 epochs	8 epochs
Time per epoch	~11 min	~13 min
Best epoch	38 of 40	15 of 15

6.1 Stage 1 - frozen trunk

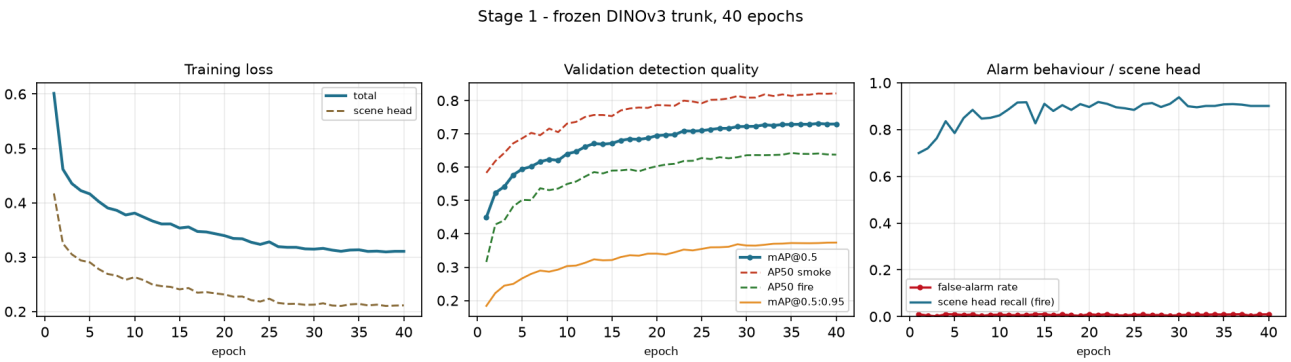

Figure 4 - Stage 1 over 40 epochs. Validation mAP@0.5 rose from 0.4494 to 0.7312 and had clearly flattened by epoch 30.

Training converged cleanly. Between epochs 30 and 40 mAP@0.5 moved only from 0.7224 to 0.7297 while the training loss sat flat at about 0.31, so further epochs at this configuration would not have helped. The false-alarm rate stayed at or below roughly 1% throughout, which is the behaviour the design targets.

6.2 Stage 2 - last two blocks released

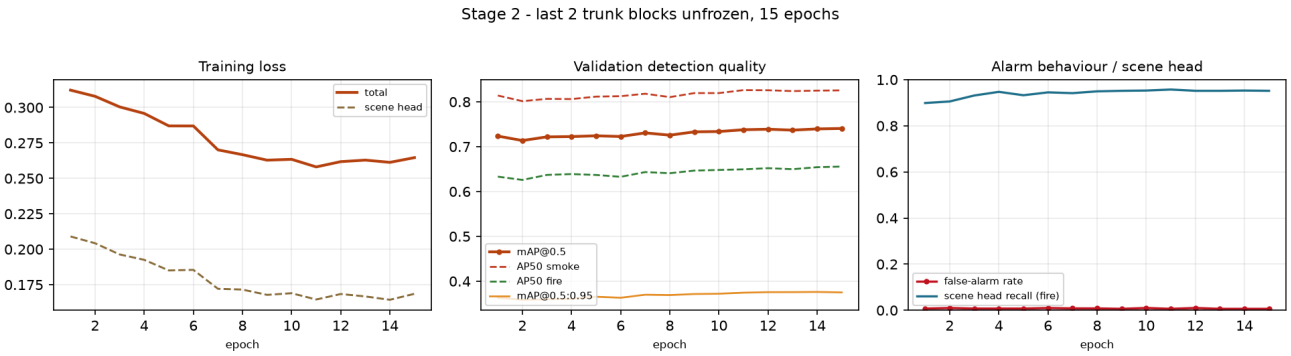

Figure 5 - Stage 2 over 15 epochs, starting from the Stage 1 weights. The best epoch is the last one, so the cosine schedule ran to completion.

Unfreezing the last two transformer blocks at a learning rate 20 times lower than the head produced a small but consistent gain. Notably the scene classifier benefited most: fire recall rose from roughly 0.90 to 0.952 at essentially unchanged false-positive rate. The image-level head gains more from adapted features than the box head does.

7. What the second stage actually bought

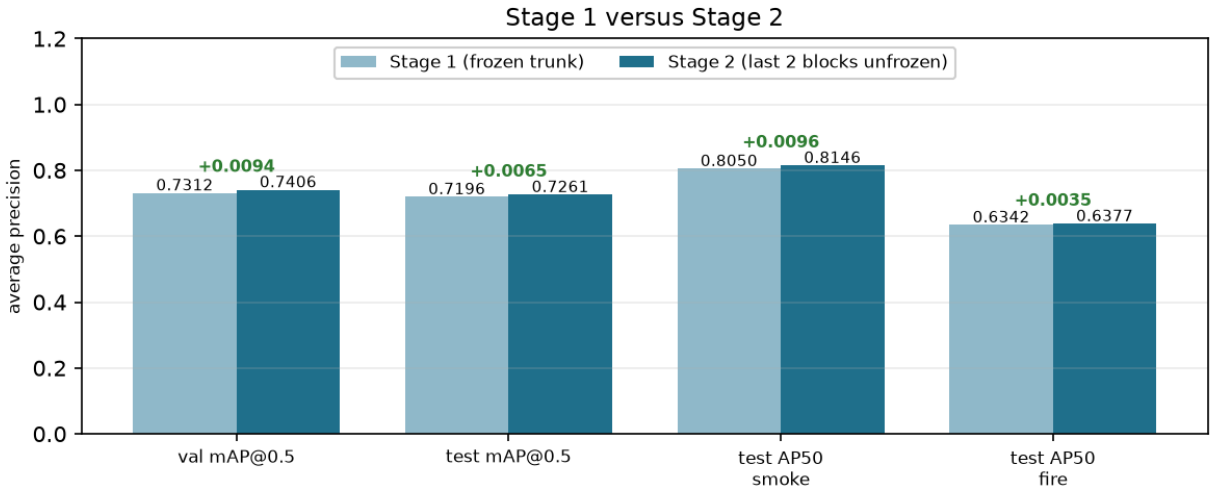

Figure 6 - Stage 1 versus Stage 2 on both validation and the held-out test split.

Metric	Stage 1	Stage 2	Change
Validation mAP@0.5	0.7312	0.7406	+0.0094
Validation AP50 - fire	0.6412	0.6556	+0.0144
Validation AP75 - fire	0.2315	0.2426	+0.0111
Test mAP@0.5	0.7196	0.7261	+0.0065
Test mAP@0.5:0.95	0.3627	0.3644	+0.0017
Scene head - fire recall @0.5	~0.900	0.952	+0.052
GPU cost	~7.3 h (40 ep)	~3.3 h (15 ep)	

Small but real - and it did not fix localisation. Fire AP75 divided by AP50 is still 0.37 after Stage 2. This is informative rather than disappointing: unfreezing improves the *features*, not the *spatial resolution*, and the result is exactly what that predicts - detection improved while tight-IoU localisation barely moved. The remaining lever is input resolution, not deeper unfreezing.

Does loose localisation matter here? Largely no. The deliverable is an alarm, not a segmentation mask; knowing there is fire and roughly where is enough to summon someone. The one place it does bite is the temporal tracker, where sloppy boxes lower frame-to-frame IoU and make track linking less reliable. The matching threshold is set to 0.20, loose enough to absorb this, but it is worth watching on real footage.

8. Test-split results and the operating point

All numbers below are from the held-out D-Fire test split (4,306 images, 2,005 verified negatives), which was never used for training or model selection. The validation-to-test gap is only 0.0145 mAP@0.5, so the model is not overfitting the validation split in any worrying way.

Class	Ground-truth boxes	AP@0.5	AP@0.75	AP@0.5:0.95
smoke	2,311	0.8146	0.4053	0.4309
fire	2,878	0.6377	0.2363	0.2980
mean		0.7261	0.3208	0.3644

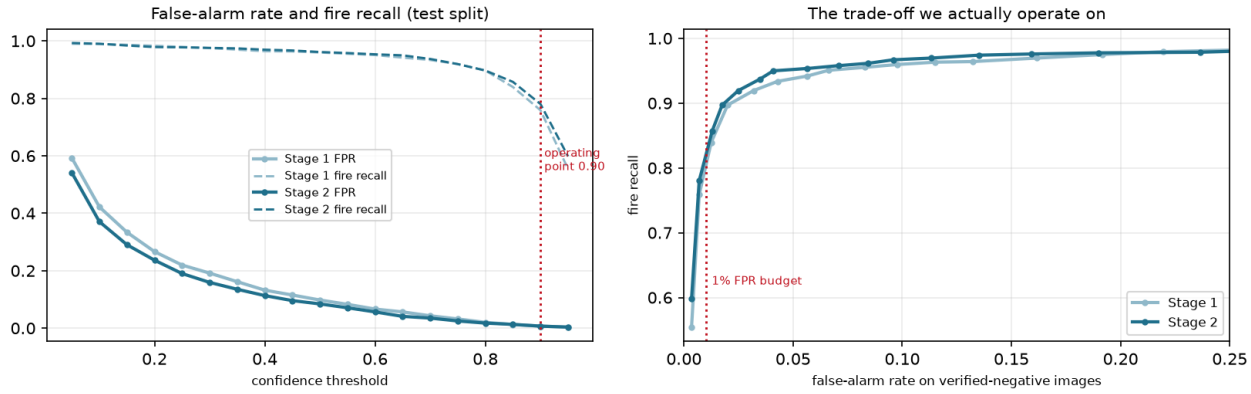

Figure 7 - Left: how the false-alarm rate and fire recall trade against the confidence threshold. Right: the same data as an operating curve. The vertical line is the 1% false-alarm budget.

Selected rows from the alarm sweep (Stage 2, test split):

Confidence	False-alarm images	False-alarm rate	Fire recall	Smoke recall
0.50	169 / 2005	8.43%	0.9614	0.9779
0.70	70 / 2005	3.49%	0.9372	0.9553
0.80	35 / 2005	1.75%	0.8969	0.9241
0.85	26 / 2005	1.30%	0.8574	0.8991
0.90	14 / 2005	0.70%	0.7803	0.8385
0.95	7 / 2005	0.35%	0.5982	0.7227

Reading this correctly. At 5 fps a 0.70% per-frame false-alarm rate is roughly 126 false boxes per hour, which no operator would tolerate. That is precisely why the video layer exists: requiring six IoU-linked detections of the same tracked object inside fifteen frames removes uncorrelated flicker entirely. In unit tests, sixty consecutive frames of high-confidence detections at random positions never raise an alarm, while a steady flame alarms within five frames and survives a three-frame occlusion.

9. Live camera test

Before site footage was available, the trained Stage 2 model was pointed at a webcam and tested against a cigarette lighter held in front of the lens, at varying flame intensity.

The model detected the lighter flame, peaking at 0.655 confidence in an earlier run and raising two temporally-confirmed alarms in the logged run below. This is a stronger result than it looks: a two-centimetre flame is far outside D-Fire's distribution of wildfires and building fires, so the frozen DINOv3 features generalised further than expected.

Frame	Time	Event	Class	Confidence	Evidence
16	19.5 s	ALARM_ON	fire	0.315	3/8 hits
24	28.3 s	ALARM_OFF	-	-	track 1
72	81.8 s	ALARM_ON	fire	0.375	3/8 hits

Two honest caveats. First, the confidences of 0.32-0.66 sit well below the 0.90 deployment threshold, so at production settings this lighter would *not* have alarmed - which is arguably correct behaviour, since a lighter is not a fire emergency. Second, the drawn box covered the flame plus its reflection on the wall, which is the loose-localisation weakness of section 7 appearing in the wild.

Two real bugs were found by this test, both now fixed. The webcam reported a frame rate of -1 through DirectShow, and the fallback expression treated that as valid, producing negative timestamps in the event log. Separately, the camera queued frames faster than roughly 1 fps CPU inference consumed them, so the detector was progressively shown older frames - what looked like lag was actually staleness. A reader thread now drains the camera and keeps only the newest frame.

10. Deployment scale

The customer specifies **70 cameras at 10 fps**, which is 700 frames per second. This is a different engineering problem from detection quality and was measured rather than estimated.

Configuration	Throughput at 640 px
Local CPU, 6 threads	0.86 fps
Kaggle P100 (derived from training timings)	~40 fps
Required	700 fps

Running every frame through the detector would need roughly seventeen P100-class GPUs. Two measurements shaped the alternative:

- **Reducing test-time proposals is free.** Cutting RPN proposals from 1000 to 300 and detections from 50 to 20 costs 0.001 mAP@0.5 and nothing at all in recall, for a roughly 1.4x faster head. This is now the default.
- **Reducing resolution is not.** Dropping 640 to 448 costs 11 points of mAP and 24 points of recall at the operating point. An apparent 2.9x speedup came almost entirely from this change and would have discarded a quarter of all detections. Resolution reduction is ruled out.

The workable architecture is a **cascade**: a cheap classical gate watches all 70 cameras and the transformer runs only on flagged frames, plus a guaranteed periodic sweep of every camera so that a gate failure can never become a silent detection failure.

Stage	Load	Hardware
Cheap gate, all cameras (frame difference 0.22 ms, glow 9 ms)	700 fps	~7 CPU cores
Guaranteed periodic sweep (70 cameras x 0.5 fps)	35 fps	

Frames passed by the gate (~2%)	14 fps	
Detector total	~49 fps	~1 GPU

One question to settle with the customer: is 10 fps an *ingest* requirement or a *detection* requirement? Fire and smoke evolve over seconds, so detecting at 1-2 fps per camera while ingesting at 10 is very likely sufficient - and it is a five- to tenfold saving in hardware.

11. Weak areas

Stated plainly, because these are what the next phase has to address.

#	Weakness	Evidence	Severity
1	Fire is much harder than smoke	Test AP50: fire 0.6377 vs smoke 0.8146	High
2	Loose localisation on fire	Fire AP75/AP50 = 0.37; unfreezing did not fix it	Medium
3	Recall cost of a low false-alarm rate	At 0.70% FPR, fire recall is only 0.78 - roughly one fire in five missed per frame before temporal confirmation recovers so	High
4	Occluded-fire capability is unvalidated	Trained on synthetic occlusions only; no real hidden-fire footage exists in the dataset	High
5	Glow prior is useless on IR cameras	Every colour-derived feature measures at exactly chance (AUC 0.500) on grayscale	Medium
6	Dataset shortcut risk	Fire images in D-Fire are systematically darker; three exposure features were removed, but the DINOv3 tokens still encod	Medium
7	No video-level validation yet	Every number is per-frame on stills; the temporal layer is validated only by unit tests	High
8	Throughput gap	700 fps required, ~40 fps per GPU measured	High
9	Domain gap	Web imagery, not elevated industrial views	High

12. Future work

Priority	Task	Why now
1	Video evaluation on the KMU Fire & Smoke dataset	Its "flame-like moving object" category is real false-positive footage. Gives the first video-level false-alarm
2	Collect and annotate site footage, then fine-tune	The single largest expected gain. Closes the domain gap that augmentation only narrows.
3	Implement the cascade service	Turns a 17-GPU requirement into roughly one. Needed before any pilot.
4	Train at 768 px	The only remaining lever for the AP75 localisation weakness, now that unfreezing has been shown not t
5	Export to ONNX / TensorRT with fp16	Typically a further two- to threefold speedup on the same hardware.
6	Capture real occluded-fire footage	The only way to validate requirement 3 properly rather than by construction.
7	Per-camera ignore masks	Furnaces, welding bays and flare stacks are permanently fire-like; masking them is the cheapest false-p
8	Revisit the smaller DINOv3 trunk (ViT-Ti, 5.5M params)	Only if the cascade proves insufficient - it would cost accuracy.

Recommended immediate next step. Items 1 and 2 together. The model is good enough that further training on D-Fire has clearly diminishing returns; what it now needs is evidence from real video and data from the actual deployment site.

Appendix A - Reproducing these results

Stage 1:

```
python scripts/train_dinov3_detector.py --epochs 40 --imgsz 640 --batch 8 --workers 4 --name stage1 --zip
```

Stage 2:

```
python scripts/train_dinov3_detector.py --epochs 15 --imgsz 640 --batch 8 --workers 4 --unfreeze-last-n 2
--lr 5e-5 --name stage2 --zip --init-from runs/fire_smoke/stage1/weights/best.pt
```

Test evaluation:

```
python scripts/eval_dinov3_detector.py --weights .../stage2/weights/best.pt --split test --target-fpr
0.01
```

Live camera test:

```
python scripts/predict_video_dinov3.py --weights best.pt --source 0 --show --no-save --conf 0.30 --window
8 --enter-hits 3
```

Code: `github.com/CoderIsSleeping/fire-and-smoke`. This document is generated by `scripts/build_training_report_pdf.py` from the run artifacts in `reports/data/`, so it stays consistent with what the runs actually produced.